MySQL DBMS Practical Coding Handbook

Focus Areas: DDL, DML, DQL, DCL, TCL, JOINs, Triggers & Complete Database Implementation.

1. Data Definition Language (DDL)

DDL কমান্ড দিয়ে ডাটাবেজের স্ট্রাকচার তৈরি, পরিবর্তন ও মুছে ফেলা হয় (Auto-committed)।

```
-- Create Database and Tables
CREATE DATABASE IF NOT EXISTS lab_db;
USE lab_db;

CREATE TABLE departments (
    dept_id INT AUTO_INCREMENT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE employees (
    emp_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    salary DECIMAL(10,2) CHECK (salary > 0),
    dept_id INT,
    hire_date DATE DEFAULT (CURRENT_DATE),
    CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

-- Altering and Dropping Structures
ALTER TABLE employees ADD COLUMN email VARCHAR(100) UNIQUE;
ALTER TABLE employees MODIFY COLUMN name VARCHAR(120) NOT NULL;
TRUNCATE TABLE employees; -- Resets table data and Auto-increment
```

2. Data Manipulation Language (DML)

DML কমান্ড দিয়ে ডাটা ইনসার্ট, আপডেট এবং ডিলিট করা হয়।

```
-- Insert Data
INSERT INTO departments (dept_name) VALUES ('CSE'), ('EEE'), ('BBA');

INSERT INTO employees (name, salary, dept_id) VALUES
('Rahim', 50000.00, 1),
('Karim', 60000.00, 1),
('Anita', 45000.00, 2);

-- Update and Delete with WHERE clause
UPDATE employees SET salary = 55000.00 WHERE emp_id = 1;
DELETE FROM employees WHERE emp_id = 3;
```

3. Data Query Language (DQL)

DQL (SELECT) দিয়ে ডাটাবেজ থেকে ডাটা ফিল্টার ও কোয়েরি করা হয়।

```
-- Filtering with WHERE, IN, LIKE, and Aggregations
SELECT emp_id, name, salary
FROM employees
WHERE dept_id IN (1, 2) AND salary >= 50000;

-- Counting and Grouping
SELECT dept_id, COUNT(*) AS total_emp, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING COUNT(*) >= 1;
```

4. Data Control Language (DCL)

ইউজার পারমিশন ম্যানেজ করার জন্য ব্যবহার করা হয়।

```
-- User Access Control
CREATE USER 'student'@'localhost' IDENTIFIED BY 'Pass123!';
GRANT SELECT, INSERT ON lab_db.* TO 'student'@'localhost';
SHOW GRANTS FOR 'student'@'localhost';
REVOKE INSERT ON lab_db.* FROM 'student'@'localhost';
FLUSH PRIVILEGES;
```

5. Transaction Control Language (TCL)

ACID ট্রানজাকশন পরিচালনা করার জন্য TCL ব্যবহৃত হয়।

```
-- Transaction Management
START TRANSACTION;
UPDATE employees SET salary = salary + 5000 WHERE emp_id = 1;
SAVEPOINT sp1;

UPDATE employees SET salary = salary + 2000 WHERE emp_id = 2;
-- ROLLBACK TO sp1; -- Rollback to savepoint if error occurs

COMMIT; -- Make changes permanent
```

6. SQL JOIN Operations

একাধিক টেবিলের মধ্যে সম্পর্ক তৈরি করে ডাটা তুলে আনার কোড।

```
-- INNER JOIN: matching records in both tables
SELECT e.emp_id, e.name, d.dept_name, e.salary
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;

-- LEFT JOIN: all records from left, matched from right
SELECT d.dept_name, e.name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id;

-- RIGHT JOIN & CROSS JOIN
SELECT e.name, d.dept_name FROM employees e RIGHT JOIN departments d ON e.dept_id =
d.dept_id;
SELECT e.name, d.dept_name FROM employees e CROSS JOIN departments d;
```

7. Triggers (BEFORE & AFTER)

ডাটা ইনসার্ট/আপডেটের সময় স্বয়ংক্রিয়ভাবে রান হওয়া ট্রিগার।

```
-- Audit Table Setup
CREATE TABLE emp_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT,
    action_type VARCHAR(20),
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    changed_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Trigger 1: Validation (BEFORE INSERT)
DELIMITER //
CREATE TRIGGER trg_check_salary
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 10000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be at least 10,000!';
    END IF;
END //

-- Trigger 2: Logging (AFTER UPDATE)
CREATE TRIGGER trg_audit_salary
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    IF OLD.salary <> NEW.salary THEN
        INSERT INTO emp_audit (emp_id, action_type, old_salary, new_salary)
        VALUES (NEW.emp_id, 'UPDATE', OLD.salary, NEW.salary);
    END IF;
END //
DELIMITER ;
```

8. Complete Executable Database Project

সম্পূর্ণ রান-এবল প্রজেক্ট কোড (Store Management System):

```sql
CREATE DATABASE IF NOT EXISTS store_db;
USE store_db;

CREATE TABLE categories (
    cat_id INT AUTO_INCREMENT PRIMARY KEY,
    cat_name VARCHAR(50) NOT NULL
);

CREATE TABLE products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    cat_id INT,
    price DECIMAL(10,2) NOT NULL,
    stock INT DEFAULT 0,
    FOREIGN KEY (cat_id) REFERENCES categories(cat_id)
);

-- Sample Data Insertion
INSERT INTO categories (cat_name) VALUES ('Electronics'), ('Groceries'), ('Clothing');
INSERT INTO products (name, cat_id, price, stock) VALUES
('Laptop', 1, 75000.00, 10),
('Mouse', 1, 800.00, 50),
('Rice 5kg', 2, 450.00, 100),
('T-Shirt', 3, 1200.00, 30);

-- Practical Required Queries: WHERE, IN, COUNT, JOIN, Aggregations

-- Query 1: Filtering using WHERE and IN
SELECT product_id, name, price
FROM products
WHERE cat_id IN (1, 2) AND price > 500.00;

-- Query 2: COUNT products per Category using JOIN & GROUP BY
SELECT c.cat_name, COUNT(p.product_id) AS total_items, AVG(p.price) AS avg_price
FROM categories c
LEFT JOIN products p ON c.cat_id = p.cat_id
GROUP BY c.cat_id, c.cat_name
HAVING COUNT(p.product_id) > 0;

-- Query 3: Finding products priced above overall average
SELECT name, price FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

Execution Order: Execute code blocks top-to-bottom in MySQL Workbench. Ensure parents are created before child tables referencing them via Foreign Keys.